

Lezione 10 - Multithreading con OpenMP

OPENMP

◆ Idea

👉 **OpenMP** è una API per la programmazione parallela su sistemi:

- multi-processore
- *a memoria condivisa*

👉 permette di eseguire più parti di codice *in parallelo* usando i **thread**

◆ Modello di esecuzione

👉 il modello è il seguente:

- esiste un **thread principale**
- questo può creare più thread secondari
- i thread eseguono codice *in parallelo*

✳ tutti i thread accedono alla stessa *memoria*

◆ Tipo di parallelismo

👉 OpenMP utilizza:

✳ **parallelismo a memoria condivisa**

👉 significa:

- tutti i thread vedono le stesse variabili
- possono leggere e scrivere sugli stessi dati

◆ Conseguenza importante

👉 questo modello è molto comodo, ma introduce problemi:

✳️ possibili *race condition*

👉 cioè:

- più thread modificano la stessa variabile
- risultato non prevedibile

◆ Altri modelli (confronto)

👉 esistono anche modelli diversi:

- sistemi con memoria *non condivisa*
- i processori comunicano tramite messaggi

👉 in OpenMP invece:

✳️ la comunicazione avviene tramite memoria condivisa

📌 PRAGMA OMP PARALLEL

◆ Idea

👉 il costrutto principale di OpenMP è:

```
#pragma omp parallel
```

👉 indica che il blocco di codice successivo deve essere eseguito *in parallelo*

◆ Funzionamento

👉 quando il programma incontra questa direttiva:

- il **thread principale** crea più thread

- ogni thread esegue lo stesso codice
- al termine, tutti i thread si sincronizzano

✳ il programma continua solo quando tutti i thread hanno finito

◆ Struttura

👉 il codice parallelo deve essere scritto così:



Codice:

```
#pragma omp parallel
{
    // codice eseguito da tutti i thread
}
```

👉 se ci sono più istruzioni:

- devono essere racchiuse tra { }

◆ Comportamento

👉 tutti i thread eseguono:

✳ lo stesso codice

👉 quindi:

- se non facciamo nulla
- tutti i thread fanno lo stesso lavoro

◆ Esempio concettuale

👉 struttura del programma:



Codice:

```
a();

#pragma omp parallel
{
    b();
    c();
}

d();
```

◆ Esecuzione

👉 succede questo:

- a() → eseguito dal thread principale
- b() e c() → eseguiti *in parallelo* da tutti i thread
- d() → eseguito dopo la fine della sezione parallela

◆ Idea importante

👉 OpenMP segue un modello:

✳ *fork-join*

👉 cioè:

- fork → creazione dei thread
- join → sincronizzazione finale

✳ base di tutto il funzionamento di OpenMP

VARIABILI E THREAD

◆ Idea

👉 in una sezione parallela, le variabili possono essere:

- **condivise (shared)**
- **private (locali al thread)**

✳️ capire questa differenza è fondamentale

◆ Variabili condivise

👉 le variabili definite *fuori* dalla sezione parallela:

- sono **condivise tra tutti i thread**

👉 quindi:

- tutti i thread vedono la stessa variabile
- tutti possono modificarla

✳️ rischio di *race condition*

◆ Variabili private

👉 le variabili definite *dentro* la sezione parallela:

- sono **private per ogni thread**

👉 quindi:

- ogni thread ha la sua copia
- possono essere modificate senza problemi

✳️ non sono visibili fuori dal blocco

◆ Esempio



Codice:

```
int x = 2;

#pragma omp parallel
{
    int y = 3;
}
```

◆ Spiegazione

👉 x :

- definita fuori
- è **condivisa**

👉 y :

- definita dentro
- è **privata per ogni thread**

◆ Problema principale

👉 se più thread modificano una variabile condivisa:

✳️ si verifica una *race condition*

👉 cioè:

- il risultato dipende dall'ordine di esecuzione
- comportamento non deterministico

◆ Idea importante

👉 in OpenMP:

- condividere dati è facile
- ma è pericoloso

✳️ bisogna controllare l'accesso alle variabili condivise

📌 FUNZIONI OPENMP E THREAD ID

◆ Idea

👉 per far fare cose diverse ai thread, possiamo usare funzioni OpenMP

👉 ogni thread ha:

- un identificatore (*thread ID*)
- informazioni sul numero totale di thread

✳️ possiamo usare queste informazioni per dividere il lavoro

◆ Funzioni principali

👉 fuori dalle sezioni parallele:

- `omp_get_max_threads()`
 - ritorna il numero massimo di thread utilizzabili

👉 dentro le sezioni parallele:

- `omp_get_num_threads()`
 - ritorna il numero di thread attivi
- `omp_get_thread_num()`
 - ritorna l'ID del thread corrente (da 0 a n-1)

◆ Esempio



Codice:

```
#pragma omp parallel
{
    int i = omp_get_thread_num();
    int j = omp_get_num_threads();
    b(i, j);
}
```

◆ Spiegazione

👉 ogni thread:

- ottiene il proprio *ID*
- conosce il numero totale di thread

👉 quindi:

- ogni thread può eseguire operazioni diverse
- possiamo dividere il lavoro

◆ Esempio pratico (idea)

👉 possiamo:

- dividere un array tra i thread
- ogni thread lavora su una parte

✳ parallelismo reale

◆ Numero di thread

👉 possiamo specificare quanti thread usare:



Codice:

```
#pragma omp parallel num_threads(3)
```

👉 in questo caso:

- vengono creati 3 thread

◆ Caso particolare

👉 possiamo far eseguire codice a un solo thread:



Codice:

```
#pragma omp single
```

👉 significa:

- solo un thread esegue il blocco
- gli altri lo saltano

◆ Idea importante

👉 per usare bene OpenMP:

- non basta parallelizzare
- bisogna *dividere il lavoro tra i thread*

✳️ altrimenti tutti fanno la stessa cosa



SEZIONI CRITICHE E SINCRONIZZAZIONE

◆ Problema

👉 supponiamo di avere una variabile condivisa:

- più thread vogliono modificarla

👉 esempio:

- sommare valori in una variabile comune

👉 se lo facciamo direttamente:

✳️ si verifica una *race condition*

◆ Idea

👉 dobbiamo garantire che:

✳️ solo un thread alla volta acceda a quella parte di codice

◆ Soluzione: sezione critica

👉 OpenMP fornisce:

```
#pragma omp critical
```

👉 indica che:

- il blocco di codice viene eseguito da un thread alla volta

◆ Struttura



Codice:

```
#pragma omp parallel
{
    a();

    #pragma omp critical
    {
        b();
    }

    c();
}
```

◆ Comportamento

👉 ogni thread:

- esegue `a()`
- entra nella sezione critica uno alla volta
- esegue `b()`
- continua con `c()`

👉 quindi:

✳️ la parte critica è *serializzata*

◆ Significato

👉 la sezione critica funziona come:

✳️ un *lock*

👉 cioè:

- viene acquisito all'ingresso

- rilasciato all'uscita

◆ Esempio reale



Codice:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    const int n = 1<<20;
    int * v = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        v[i] = rand() % 100;
    }
    int sum = 0;
    #pragma omp parallel
    {
        int n_thread = omp_get_thread_num();
        int slice_size = n / omp_get_num_threads();
        int start = n_thread * slice_size;
        int end = (n_thread + 1) * slice_size;
        int partial_sum = 0;
        for (int i = start; i < end; i++) {
            partial_sum += v[i];
        }
        printf("Somma parziale terminata (thread %d)\n", n_thread);
    }
    #pragma omp critical
    {
        sum += partial_sum;
    }
    printf("La somma dei valori è %d\n", sum);
    return 0;
}
```

◆ Spiegazione del codice

👉 ogni thread:

- calcola una *somma parziale* su una parte dell'array

👉 problema:

- `sum` è condivisa

👉 soluzione:

```
#pragma omp critical
{
    sum += partial_sum;
}
```

- un thread alla volta aggiorna `sum`
- evita *race condition*

◆ Attenzione

👉 le sezioni critiche:

- garantiscono correttezza
- ma riducono il parallelismo

✳ vanno usate solo quando necessario

◆ Idea importante

👉 in OpenMP:

- parallelizzare è facile

- sincronizzare è difficile

✳️ bisogna bilanciare *prestazioni* e *correttezza*

PARALLEL FOR

◆ Idea

👉 uno dei costrutti più comuni da parallelizzare è:

- il ciclo `for`

👉 OpenMP fornisce una direttiva dedicata:

```
#pragma omp parallel for
```

👉 che permette di:

✳️ dividere automaticamente le iterazioni tra i *thread*

◆ Funzionamento

👉 quando usiamo:

```
#pragma omp parallel for
for (int i = 0; i < n; i++)
{
    f(i);
}
```

👉 succede che:

- le iterazioni del ciclo vengono distribuite tra i thread
- ogni thread esegue una parte del ciclo

✳️ esecuzione *in parallelo*

◆ Esempio

👉 con 4 thread e 16 iterazioni:

- thread 0 → alcune iterazioni
- thread 1 → altre
- thread 2 → altre
- thread 3 → altre

👉 tutti lavorano contemporaneamente

◆ Equivalenza

👉 questa direttiva è equivalente a:

```
#pragma omp parallel
#pragma omp for
```

👉 cioè:

- parallel → crea i thread
- for → divide il lavoro

◆ Attenzione (IMPORTANTISSIMO)

👉 NON è la stessa cosa scrivere:

```
#pragma omp parallel
for (int i = 0; i < n; i++)
{
    f(i);
}
```

◆ Perché?

👉 in questo caso:

- ogni thread esegue l'intero ciclo

👉 quindi:

✳ tutte le iterazioni vengono ripetute da ogni thread

◆ Conseguenza

👉 invece di parallelizzare:

- moltiplichiamo il lavoro

✳ errore gravissimo

◆ Esempio reale



Codice:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int sum = 0;
    #pragma omp parallel for
    for (int i = 0; i < 10; i++)
    {
        printf("%d\n", i);
    }
    return 0;
}
```

◆ Spiegazione del codice

👉 le iterazioni del ciclo:

- vengono divise tra i thread

👉 quindi:

- ogni valore di `i` viene stampato una sola volta
- ma l'ordine non è garantito

✳️ esecuzione *in parallelo*

◆ Idea importante

👉 per parallelizzare un ciclo:

✳️ usare sempre `parallel for`

👉 evitare:

✳️ `parallel` + ciclo `for` normale

◆ Conclusione

👉 `parallel for` :

- è semplice
- è efficiente
- è il modo standard per parallelizzare cicli

✳️ uno dei costrutti più importanti di OpenMP

📌 PARALLEL FOR E NOWAIT

◆ Problema

👉 quando usiamo:

```
#pragma omp parallel for
```

👉 alla fine del ciclo:

✳ tutti i thread devono *aspettare* che gli altri thread finiscano

👉 questo comportamento è una:

✳ *barriera implicita*

◆ **Conseguenza**

👉 anche se un thread ha finito prima:

- deve aspettare gli altri

✳ possibile perdita di prestazioni

◆ **Soluzione: nowait**

👉 possiamo evitare l'attesa usando:

```
#pragma omp for nowait
```

👉 questo significa:

- i thread NON aspettano
- appena finiscono continuano

◆ **Struttura**

👉 tipicamente si usa così:



Codice:

```
#pragma omp parallel
{
    #pragma omp for nowait
    for (int i = 0; i < n; i++)
    {
        f(i);
    }

    g();
}
```

◆ Comportamento

👉 senza nowait :

- tutti i thread finiscono il ciclo
- poi eseguono g()

👉 con nowait :

- ogni thread, appena finisce
- esegue subito g()

✳ maggiore parallelismo

◆ Esempio reale



Codice:

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int sum = 0;
    #pragma omp parallel
    {
        #pragma omp for nowait
        for (int i = 0; i < 10; i++)
        {
            printf("%d\n", i);
        }
        printf("Fuori dal ciclo\n");
    }
    return 0;
}
```

◆ Spiegazione del codice

👉 il ciclo for :

- viene eseguito *in parallelo*

👉 grazie a nowait :

- un thread può stampare "Fuori dal ciclo"
- anche se altri stanno ancora eseguendo il ciclo

★ esecuzione sovrapposta

◆ **Attenzione**

👉 usare `nowait` solo se:

- non ci sono dipendenze tra le operazioni

👉 altrimenti:

✳️ rischio di errori logici

◆ **Idea importante**

👉 `nowait` :

- migliora le prestazioni
- elimina attese inutili

✳️ ma richiede attenzione

📌 **SCHEDULING DELLE ITERAZIONI**

◆ **Idea**

👉 quando usiamo `parallel for` :

- le iterazioni del ciclo devono essere distribuite tra i thread

👉 questa distribuzione è chiamata:

✳️ *scheduling*

◆ **Default (statico)**

👉 di default OpenMP usa:

✳️ *schedule statico* (default)

👉 significa:

- le iterazioni vengono assegnate in modo deterministico
- ogni thread riceve una parte del lavoro

◆ Esempio (static,1)



Codice:

```
#pragma omp parallel
{
#pragma omp for schedule(static,1)
    for (int i = 0; i < 12; i++)
    {
        f(i);
    }
}
```

◆ Significato

👉 con `static,1`:

- ogni thread prende una iterazione alla volta
- distribuzione *round-robin*

👉 esempio:

- thread 0 → i = 0, 4, 8
- thread 1 → i = 1, 5, 9
- thread 2 → i = 2, 6, 10
- thread 3 → i = 3, 7, 11

◆ Statico con blocchi

👉 possiamo usare blocchi più grandi:

```
schedule(static,2)
```

👉 significa:

- ogni thread prende 2 iterazioni consecutive

👉 esempio:

- thread 0 $\rightarrow$ (0,1), (8,9)
- thread 1 $\rightarrow$ (2,3), (10,11)
- ecc.

◆ Problema

👉 se le iterazioni hanno costo diverso:

- alcuni thread finiscono prima
- altri restano occupati

✳️ squilibrio di carico

◆ Soluzione: schedule dinamico

👉 possiamo usare:

```
schedule(dynamic,1)
```

👉 comportamento:

- un thread prende un blocco
- quando finisce, prende il prossimo disponibile

✳️ migliore bilanciamento

◆ Vantaggio

👉 il *dynamic scheduling*:

- riduce lo squilibrio
- migliora l'utilizzo dei thread

◆ Svantaggio

👉 lo scheduling dinamico:

- introduce overhead
- è più costoso

✳️ va usato solo quando necessario

◆ Idea importante

👉 scelta dello scheduling:

- *statico* → veloce, ma meno flessibile
- *dinamico* → più bilanciato, ma più costoso

✳️ è un compromesso tra prestazioni e bilanciamento

📌 OSSERVAZIONI FINALI

◆ Problemi aperti

👉 anche con OpenMP rimangono alcune difficoltà:

- parallelizzare *loop innestati*
- gestire lavori più complessi
- evitare errori di sincronizzazione

◆ Task-based parallelism

👉 un approccio alternativo è:

✳ *parallelismo basato su task*

👉 idea:

- definire una lista di lavori
- i thread prendono il prossimo lavoro disponibile

👉 vantaggio:

- maggiore flessibilità

◆ Accumulo dei risultati

👉 spesso:

- più thread producono risultati parziali
- devono essere combinati

👉 problema:

- gestione corretta delle variabili condivise

✳ rischio di *race condition*

◆ Altri strumenti

👉 OpenMP offre anche:

- operazioni *atomiche*
- controllo del modello di memoria

👉 servono per:

- gestire accessi concorrenti
- migliorare le prestazioni

◆ Idea fondamentale

👉 con OpenMP:

- parallelizzare è semplice
- ma scrivere codice corretto è difficile

◆ Conclusione

👉 per usare bene OpenMP bisogna:

- capire il modello a *thread*
- gestire le variabili condivise
- usare correttamente sincronizzazione e scheduling

✳ equilibrio tra *prestazioni* e *correttezza*